

Relatório MC521 - Percursos em Grafos

Sofia Valverde Villas Bôas - 252974

May 14, 2026

H - Course Schedule

Nesse problema, temos n matérias e m restrições. Cada restrição é da forma (u, v) , significando que a matéria u é pré-requisito da v . Queremos obter uma ordem de matérias que satisfaça todas as restrições.

Solução

A ideia principal da solução consiste em encontrar uma ordenação topológica do grafo formado pelas matérias (vértices) e pelos pré-requisitos (arestas). Para isso, primeiro foi necessário checar se havia ciclos no grafo, pois se houvesse, haveria conflito entre os pré-requisitos e o problema seria impossível.

Para checar a existência de ciclos, utilizei uma DFS que retorna true caso encontre um ciclo, e false caso contrário. Bastou checar se há back edges durante a DFS com o vetor *path*.

Se não tinha ciclos, então usei o algoritmo de Kahn para obter a ordenação topológica. Resumidamente, ele primeiro obtém os graus de entrada para cada vértice, depois inserimos em uma fila os vértices que tem grau de entrada 0. Com isso, rodamos um loop semelhante ao de uma BFS, onde vamos processando os vértices que são "fonte" (grau 0) e vamos reduzindo o grau de entrada dos vizinhos. Se um vizinho chegar a grau 0, ele é adicionado na fila. Cada vértice que retiramos da fila é adicionado no vetor da ordenação topológica.

Por fim, bastou imprimir os elementos do vetor *order*. Vale mencionar, que percebi depois que eu nem precisaria ter rodado a DFS, já que se ao final do algoritmo de Kahn, o tamanho do vetor *order* fosse diferente de n , significaria que teria um ciclo no grafo.

C - Longest Flight Route

Temos n cidades e m voos. Uolevi quer ir da cidade 1 até a cidade n , visitando o maior número de cidades possível. O problema garante que o grafo é um DAG. Resumidamente, queremos obter o caminho de maior tamanho de 1 até n no grafo.

Solução

Primeiro, devemos checar se existe caminho da cidade 1 até a cidade n . Caso contrário, já sabemos que a resposta é "IMPOSSIBLE". Para isso, bastou fazer uma dfs partindo do vértice 1 e verificar se conseguimos chegar em n .

Essa dfs também permitiu obter uma ordenação topológica, necessária para que fosse possível computar o caminho mais longo usando DP. Já que precisamos que o estado do vértice u já tenha sido computado quando fôssemos computar o estado de v .

Assim, temos:

- **Estado:** $dp[v]$ representa o número máximo de cidades visitadas em um caminho que inicia na cidade 1 e termina na cidade v .
- **Caso Base:** $dp[1] = 1$, pois o trajeto que começa e termina na cidade inicial visita apenas ela mesma. Os demais estados são inicializados com 0.
- **Transição de Estado:** Para calcular o caminho mais longo até v , basta analisarmos todos os vértices u que possuem um voo direto para v . O maior caminho até v passando por u será exatamente o maior caminho até u mais 1. Por isso, tomamos o máximo entre todas as possibilidades de chegada.

No código, ao visitarmos o vértice u (que, pela ordenação topológica, já vai ter seu $dp[u]$ calculado), iteramos pelos seus vizinhos v e tentamos relaxar a aresta: se estender o caminho de u oferecer um trajeto maior do que o que já conhecemos para v , atualizamos $dp[v] = dp[u] + 1$ e registramos u no vetor *parents* de v para, ao final, podermos reconstruir a rota.